

What is a framework

- A framework is a set of **classes** and **interfaces** which provide a ready-made architecture.
- A collection framework is a unified architecture for representing and manipulating collections.
- Collection Framework: represents a group of objects as a single entity in the Java programming language. we need classes and interfaces defined by the Collection Framework



Collection framework

All collections frameworks(`java.util`) contain the following:

- **Interfaces:** These are abstract data types that represent collections. Interfaces allow collections to be manipulated independently of the details of their representation. In object-oriented languages, interfaces generally form a hierarchy.
- **Implementations:** These are the concrete implementations of the collection interfaces. In essence, they are reusable data structures.
- **Algorithms:** These are the methods that perform useful computations, such as searching and sorting, on objects that implement collection interfaces. The algorithms are said to be polymorphic: that is, the same method can be used on many different implementations of the appropriate collection interface. In essence, algorithms are reusable functionality.



Benefits of the java Collection Framework

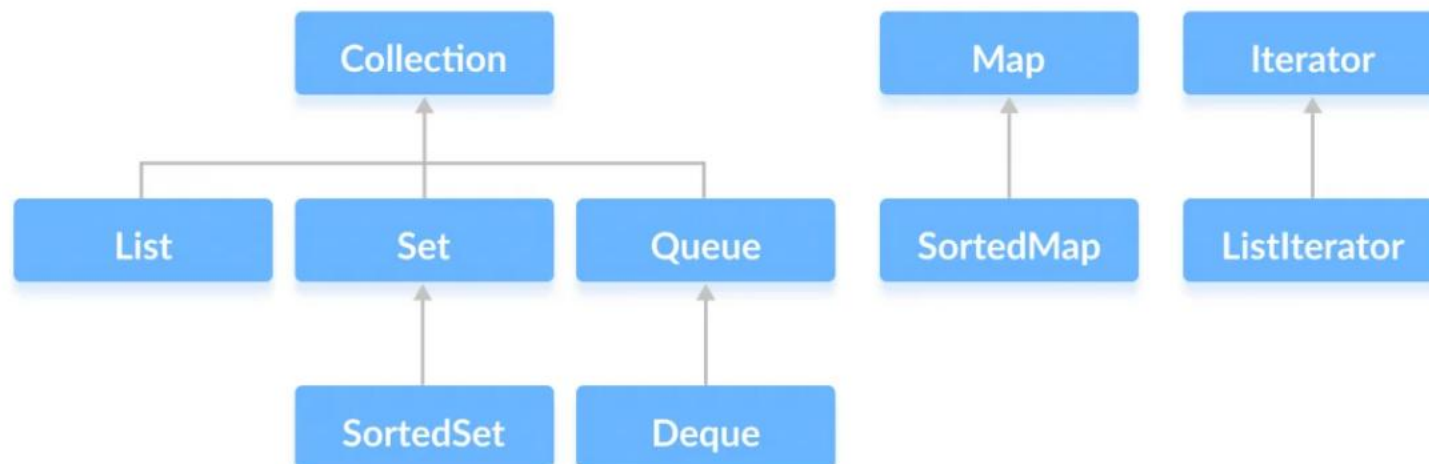
- 1.Reduces programming effort** by providing data structures and algorithms so you don't have to write them yourself.
- 2.Increases performance** by providing high-performance implementations of data structures and algorithms. Because the various implementations of each interface are interchangeable, programs can be tuned by switching implementations.
- 3.Provides interoperability** between unrelated APIs by establishing a common language to pass collections back and forth.
- 4.Reduces the effort required to learn APIs** by requiring you to learn multiple ad hoc collection APIs.
- 5.Reduces the effort required to design and implement APIs** by not requiring you to produce ad hoc collections APIs.
- 6.Fosters software reuse** by providing a standard interface for collections and algorithms with which to manipulate them.



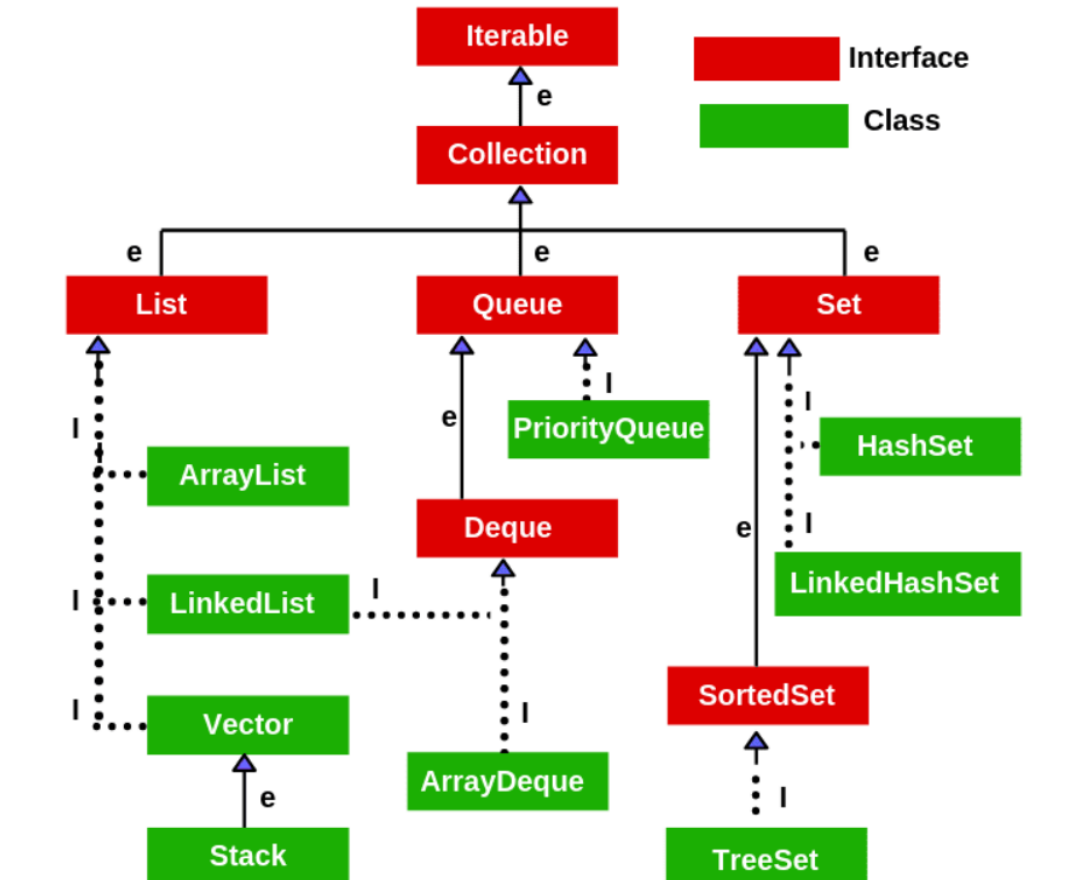
Hierarchy of Collection framework

- The collection interfaces are divided into two groups:
 - *The most basic interface, [java.util.Collection](#).
 - *The other collection interfaces are based on *java.util.Map* and are not true collections

Java Collections Framework



Hierarchy of Collection framework



e → extends, I → implements

Hierarchy of Collection framework

- **Iterable Interface:** This is the root interface for the entire collection framework. The collection interface extends the iterable interface. Therefore, inherently, all the interfaces and classes implement this interface. The main functionality of this interface is to provide an iterator for the collections. Therefore, this interface contains only one abstract method which is the iterator
- **Collection Interface:** This interface extends the iterable interface and is implemented by all the classes in the collection framework. This interface contains all the basic methods which every collection has like adding the data into the collection, removing the data, clearing the data, etc.



Hierarchy of Collection framework

- **Iterable Interface:** This is the root interface for the entire collection framework. The collection interface extends the iterable interface. Therefore, inherently, all the interfaces and classes implement this interface. The main functionality of this interface is to provide an iterator for the collections. Therefore, this interface contains only one abstract method which is the iterator
- **Collection Interface:** This interface extends the iterable interface and is implemented by all the classes in the collection framework. This interface contains all the basic methods which every collection has like adding the data into the collection, removing the data, clearing the data, etc.



List Interface

- It is a child interface of the collection interface.
- This interface is dedicated to the data of the list type in which we can store all the ordered collections of the objects. This deals with the index or position-specific functions like getting an element or setting an element. It deals with the arrays and lists types of operations like ArrayList, LinkedList, Vector, and Stack.



Set Interface

- Set in Java is an **interface** declared in java.util package. It extends the collection interface that allows creating an unordered collection or list, where duplicate values are not allowed. As the name implies, a set in **Java** is used to create a mathematical set.
- Since the set extends the collection interface, it does not allow duplicate elements. In the hierarchy, NavigableSet and SortedSet are the two interfaces that extend set in Java.
- Implementation classes of Set interface are:

HashSet

LinkedHashSet

EnumSet

TreeSet



Queue Interface

- Java Queue is an interface available in java.util package and extends java.util.Collection interface.
- Just like Java List, Java Queue is a collection of ordered elements (Or objects) but it performs insert and remove operations differently. We can use Queue to store elements before processing those elements.
- Java Queue follows FIFO order to insert and remove it's elements. FIFO stands for First In First Out
- Implementation classes are: ArrayDeque, LinkedList and PriorityQueue
- Interfaces that extends Queue interface are : Deque, BlockingDeque and BlockingQueue



Queue Interface

- Java Queue is an interface available in java.util package and extends java.util.Collection interface.
- Just like Java List, Java Queue is a collection of ordered elements (Or objects) but it performs insert and remove operations differently. We can use Queue to store elements before processing those elements.
- Java Queue follows FIFO order to insert and remove it's elements. FIFO stands for First In First Out
- Implementation classes are: ArrayDeque, LinkedList and PriorityQueue
- Interfaces that extends Queue interface are : Deque, BlockingDeque and BlockingQueue



Map Interface

- Map in Java is an interface available in java.util package and it stores the data in key and value pairs. It does not allow duplicate keys. The map interface in Java is often misunderstood as being a subtype of the Collections interface. But that is not true, and hence, the Java map functions differently from the Collection interface
- There are three classes to implement maps. These three classes are HashMap, LinkedHashMap, and TreeMap
- The SortedMap interface extends the Map interface



Example of Collection

- In the example, we used **raw** version of the collection classes
- They store every item as an **object** reference variable

```
1 package y2Collection;
2 import java.util.*;
3
4 public class Y2CollectionMain {
5
6     public static void main(String[] args) {
7         // TODO Auto-generated method stub
8         Collection value=new ArrayList();
9         value.add(3);
10        value.add(6);
11
12        for (Object obj:value) {
13            int n=(Integer)obj;
14            System.out.println(n);
15        }
16    }
17 }
18 }
19 }
```



Generics

- **Generics** means **parameterized types**. The idea is to allow type (Integer, String, ... etc., and user-defined types) to be a parameter to methods, classes, and interfaces. Using Generics, it is possible to create classes that work with different data types. An entity such as class, interface, or method that operates on a parameterized type is a generic entity
- Generics was added in Java 5 to provide compile-time type checking and removing risk of ClassCastException that was common while working with collection classes. The whole collection framework was re-written to use generics for type-safety. Let's see how generics help us using collection classes safely



Need of Generics

- Let's imagine a scenario where we want to create a list in Java to store Integer.
- We might try to write the following:

```
List list = new LinkedList();  
list.add(new Integer(1));  
Integer i = list.iterator().next();
```

- Surprisingly, the compiler will complain about the last line. It doesn't know what data type is returned.
- The compiler will require an explicit casting:

```
Integer i = (Integer) list.iterator().next();
```



Need of Generics

- No contract could guarantee that the return type of the list is an Integer. The defined list could hold any object. When looking at types, it can only guarantee that it is an Object and therefore requires an explicit cast to ensure that the type is safe.
- This cast can be annoying — we know that the data type in this list is an Integer. The cast is also cluttering our code. It can cause type-related runtime errors if a programmer makes a mistake with the explicit casting.
- It would be much easier if programmers could express their intention to use specific types and the compiler ensured the correctness of such types. This is the core idea behind generics.



Need of Generics

- Let's modify the first line of the previous code snippet:

```
List<Integer> list = new LinkedList<>();
```

- By adding the diamond operator <> containing the type, we narrow the specialization of this list to only Integer type. In other words, we specify the type held inside the list. The compiler can enforce the type at compile time.
- In small programs, this might seem like a trivial addition. But in larger programs, this can add significant robustness and makes the program easier to read.



Advantages of Generics

- 1. Code Reuse:** We can write a method/class/interface once and use it for any type we want.
- 2. Type Safety:** Generics make errors to appear compile time than at run time (It's always better to know problems in your code at compile time rather than making your code fail at run time). Suppose you want to create an ArrayList that store name of students, and if by mistake the programmer adds an integer object instead of a string, the compiler allows it. But, when we retrieve this data from ArrayList, it causes problems at runtime.



Java Generic class

- We can define our own classes with generics type. A generic type is a class or interface that is parameterized over types. We use angle brackets (<>) to specify the type parameter. To understand the benefit, let's say we have a simple class as:

```
package y2Collection;

public class Printer<T> {
    T anything;

    public Printer(T anything) {
        super();
        this.anything = anything;
    }

    public T getAnything() {
        return anything;
    }

    public void setAnything(T anything) {
        this.anything = anything;
    }

    public void display() {
        System.out.println(this.anything);
    }
}
```

```
1 package y2Collection;
2 import java.util.*;
3
4 public class Y2CollectionMain {
5
6     public static void main(String[] args) {
7         Printer<Integer> p=new Printer<>(3);
8         p.display();
9         Printer<String> p1=new Printer<>("hello");
10        p1.display();
11    }
12 }
13
14
```



Java Generic Type

- Java Generic Type Naming convention helps us understanding code easily and having a naming convention is one of the best practices of Java programming language. So generics also comes with its own naming conventions. Usually, type parameter names are single, uppercase letters to make it easily distinguishable from java variables. The most commonly used type parameter names are:
- E - Element (used extensively by the Java Collections Framework, for example ArrayList, Set etc.)
- K - Key (Used in Map)
- N - Number
- T - Type
- V - Value (Used in Map)
- S,U,V etc. - 2nd, 3rd, 4th types



Java Generic method

We write generic methods with a single method declaration, and we can call them with arguments of different types. The compiler will ensure the correctness of whichever type we use.

- These are some properties of generic methods:
- Generic methods have a type parameter (the diamond operator enclosing the type) before the return type of the method declaration.
- Type parameters can be bounded .
- Generic methods can have different type parameters separated by commas in the method signature.
- Method body for a generic method is just like a normal method.



Java Generic method

```
public class GenericsMain {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        String name=(String)printAnything("Nana","Ingabire",17);  
        System.out.println("the name is"+name);  
    }  
    public static <T,G>T printAnything(T firstName,T lastName, G age){  
        return firstName ;  
    }  
}
```

parameters

Return type



Java Generics bounded type parameters

- Suppose we want to restrict the type of objects that can be used in the parameterized type, To declare a bounded type parameter, list the type parameter's name, followed by the extends keyword, followed by its upper bound, similar like below method.

```
    {  
    public static<T extends Computer> void anything(T value) {  
        System.out.println(value);  
    }  
}
```



Upper bound

- In Java generics, the term "upper bound" refers to a constraint that you can place on a type parameter. An upper bound specifies that the type argument must be a subtype of a particular type. It is denoted using the extends keyword

```
public class Box<T extends Number> {  
    private T value;  
  
    public Box(T value) {  
        this.value = value;  
    }  
  
    public T getValue() {  
        return value;  
    }  
  
    public void setValue(T value) {  
        this.value = value;  
    }  
}
```

- In this example, the generic type parameter T is constrained to be a subtype of Number. This means that when you instantiate the Box class with a type argument, it must be a type that extends or implements the Number class or interface



Upper bound

```
public class GenericsMain {  
    public static void main(String[] args) {  
        Box<Integer> intBox = new Box<>(42); // This is valid because Integer extends Number  
        Box<String> strBox = new Box<>("Hello"); // This would result in a compilation error  
    }  
}
```

- In the example above, `Box<Integer>` is valid since `Integer` is a subtype of `Number`, but `Box<String>` would result in a compilation error because `String` does not extend `Number`.



Upper bound wildcard

- In Java, generics provide a way to create classes, interfaces, and methods that can work with different types while providing compile-time type safety. Wildcards are a feature in generics that allow you to represent an unknown type or a family of types.
- An upper-bounded wildcard is denoted by the `?` extends `T` syntax, where `T` is a type. It restricts the unknown type to be a subtype of `T` (including `T` itself). Upper-bounded wildcards are useful when you want to accept a range of related types without specifying a concrete type.
- In the example above, `Box<Integer>` is valid since `Integer` is a subtype of `Number`, but `Box<String>` would result in a compilation error because `String` does not extend `Number`.



Upper bound wildcard

```
5 public class WildcardExample {  
5     // Method that accepts a List of objects of a type that extends Number  
7     public static double sumOfList(List<? extends Number> list) {  
3         double sum = 0.0;  
3         for (Number number : list) {  
3             sum += number.doubleValue();  
1         }  
2         return sum;  
3     }  
4  
5     public static void main(String[] args) {  
5         // Example with a List of Integers  
7         List<Integer> integerList = Arrays.asList(12,2,4,5,89);  
3         double sumOfIntegers = sumOfList(integerList);  
3         System.out.println("Sum of integers: " + sumOfIntegers);  
3  
1         // Example with a List of Doubles  
2         List<Double> doubleList = Arrays.asList(1.5, 2.5, 3.5, 4.5, 5.5);  
3         double sumOfDoubles = sumOfList(doubleList);  
4         System.out.println("Sum of doubles: " + sumOfDoubles);  
5     }  
5 }  
7 }
```



Lower bound

- In Java generics, the "lower bound" is a constraint that you can place on a type parameter, specifying that the type argument must be a supertype of a particular type. The lower bound is denoted using the **super** keyword.

```
public class Box<T super Number> {  
    private T value;  
  
    public Box(T value) {  
        this.value = value;  
    }  
  
    public T getValue() {  
        return value;  
    }  
  
    public void setValue(T value) {  
        this.value = value;  
    }  
}
```

- However, the syntax above is not valid in Java. In Java, lower bounds are typically used with wildcard types, not directly on type parameters. Here's a corrected example using a wildcard:



Lower bound wildcard

- In this example, the **'processWildcard'** method takes a **'Box'** with a wildcard type parameter that is a supertype of **'Number'**. This allows the method to work with **Box<Object>**, **Box<Number>**, or **Box<Object>**.

```
public class Box<T> {  
    private T value;  
  
    public Box(T value) {  
        this.value = value;  
    }  
  
    public T getValue() {  
        return value;  
    }  
  
    public void setValue(T value) {  
        this.value = value;  
    }  
  
    @Override  
    public String toString() {  
        return "Box [value=" + value + "]";  
    }  
  
    public void processWildcard(Box<? super Number> box) {  
        // Some code that works with box, where the type parameter is a supertype of Number  
        System.out.println(box);  
    }  
}
```



Lower bound wildcard

- In this example, the **'processWildcard'** method takes a **'Box'** with a wildcard type parameter that is a supertype of **'Number'**. This allows the method to work with **Box<Object>**, **Box<Number>**, or **Box<Object>**.

```
public class Box<T> {  
    private T value;  
  
    public Box(T value) {  
        this.value = value;  
    }  
  
    public T getValue() {  
        return value;  
    }  
  
    public void setValue(T value) {  
        this.value = value;  
    }  
  
    public void processWildcard(Box<? super Number> box) {  
        // Some code that works with box, where the type parameter is a supertype of Number  
    }  
}
```



Lower bound wildcard

```
1  
2 public class GenericsMain {  
3  
4     public static void main(String[] args) {  
5         Box<Object> objBox = new Box<>("Hello");  
6         Box<Number> numBox = new Box<>(42);  
7  
8         objBox.processWildcard(objBox); // Valid  
9         objBox.processWildcard(numBox); // Valid  
0     }  
1  
2 }
```

```
Box [value=Hello]  
Box [value=42]
```



Primitive data types and Generics in Java

- In Java, generics primarily operate with reference types, and primitive types (like **int**, **char**, **double**, etc.) cannot be directly used as type arguments. This is because generics in Java work with objects, and primitive types are not objects.
- However, Java provides wrapper classes for each primitive type, and these wrapper classes can be used with generics. The wrapper classes include:
 - **Integer** for int
 - **Double** for double
 - **Character** for char
 - **Boolean** for boolean
 - and so on..



Primitive data types and Generics in Java

```
public class Box<T> {  
    private T value;  
  
    public Box(T value) {  
        this.value = value;  
    }  
  
    public T getValue() {  
        return value;  
    }  
  
    public void setValue(T value) {  
        this.value = value;  
    }  
}
```

```
public class GenericsMain {  
  
    public static void main(String[] args) {  
        // Using Box with Integer (wrapper for int)  
        Box<Integer> intBox = new Box<>(42);  
        int value = intBox.getValue(); // Auto-unboxing from Integer to int  
    }  
}
```



Primitive data types and Generics in Java

- In this example, the **Box** class is generic, and it can be used with the wrapper class **Integer**. Java automatically handles the conversion between primitive types and their corresponding wrapper classes through a process called **autoboxing** (converting a primitive type to its wrapper type) and **auto-unboxing** (converting a wrapper type to its primitive type).
- Starting from Java 5, generics were introduced to provide type safety and eliminate the need for explicit casting. While generics work well with wrapper types, they are not directly compatible with primitive types due to the object-oriented nature of generics in Java.



List Interface

- The List interface is found in java.util package and inherits the Collection interface. It is a factory of the ListIterator interface. Through the ListIterator, we can iterate the list in forward and backward directions. The implementation classes of the List interface are ArrayList, LinkedList, Stack, and Vector



Operations in List Interface

- **Operation 1:** Adding elements to List class using add() method
- **Operation 2:** Updating elements in List class using set() method
- **Operation 3:** Searching for elements using indexOf(), lastIndexOf methods
- **Operation 4:** Removing elements using remove() method
- **Operation 5:** Accessing Elements in List class using get() method
- **Operation 6:** Checking if an element is present in the List class using contains() method



Example of different operations in List Interface

```
public class ExampleMain {  
    public static void main(String[] ar) {  
        List<String> names=new ArrayList<>();  
        names.add("Larisse");  
        names.add("Skylar");  
        names.add("Sam");  
        names.add("Lucas");  
        // adding element between Sam and Lucas  
        names.add(3,"Joanna");  
        System.out.println(names);  
        // updating element in the list  
        names.set(1, "Skylar Irisa");  
        System.out.println(names.get(1));  
        //remove element from the list  
        names.remove(0);  
        System.out.println(names);  
        //to check if the list contains Larisse  
        System.out.println("do we have Larisse?" + names.contains("Larisse"));  
        List<String> newNames=new ArrayList<>();  
        newNames.add("Chesi");  
        newNames.add("Maelisse");  
        newNames.add("Asa");  
        names.addAll(newNames); //adding a list  
        System.out.println(names);  
    }  
}
```



Vector Class

- Vector is a class that is implemented in the collection framework implements a growable array of objects. Vector implements a dynamic array that means it can grow or shrink as required. Like an array, it contains components that can be accessed using an integer index. Vectors basically fall in legacy classes but now it is fully compatible with collections. Let's see how to create a list object using this class.
- They are very similar to ArrayList, but Vector is synchronized and has some legacy methods that the collection framework does not contain.
- It also maintains an insertion order like an ArrayList. Still, it is rarely used in a non-thread environment as it is **synchronized**, and due to this, it gives a poor performance in adding, searching, deleting, and updating its elements.



Vector example

```
public class ExampleMain {  
    public static void main(String[] ar) {  
        // Size of the vector  
        int n = 5;  
        List<Integer> v=new Vector<>();  
        for (int i = 1; i <= n; i++)  
            v.add(i);  
  
        // Printing elements  
        System.out.println(v);  
  
        // Remove element at index 3  
        v.remove(3);  
  
        // Displaying the list after deletion  
        System.out.println(v);  
  
        // Printing elements one by one  
        for (int i = 0; i < v.size(); i++)  
            System.out.print(v.get(i) + " ");  
    }  
}
```



Stack Class

- Stack is a class that is implemented in the collection framework and extends the vector class models and implements the Stack data structure. The class is based on the basic principle of last-in-first-out. In addition to the basic push and pop operations, the class provides three more functions of empty, search and peek.



Stack example

```
import java.util.*;

public class ExampleMain {
    public static void main(String[] ar) {

        Stack<Integer> stack = new Stack<>();
        stack_push(stack);
        stack_pop(stack);
        stack_push(stack);
        stack_peek(stack);
        stack_search(stack, 2);
        stack_search(stack, 6);
        stack.add(10); //you can use add/methods from list interface
        System.out.println(stack);
    }

    //Pushing element on the top of the stack
    static void stack_push(Stack<Integer> stack)
    {
        for(int i = 0; i < 5; i++)
        {
            stack.push(i);
        }
    }

    // Popping element from the top of the stack
    static void stack_pop(Stack<Integer> stack)
    {
        System.out.println("Pop Operation:");

        for(int i = 0; i < 5; i++)
        {
            Integer y = (Integer) stack.pop();
            System.out.println(y);
        }
    }

    // Searching element in the stack
    static void stack_search(Stack<Integer> stack, int element)
    {
        Integer pos = (Integer) stack.search(element);

        if(pos == -1)
            System.out.println("Element not found");
        else
            System.out.println("Element is found at position: " + pos);
    }
}
```

LinkedList

- LinkedList is a class that is implemented in the collection framework which inherently implements the linked list data structure. It is a linear data structure where the elements are not stored in contiguous locations and every element is a separate object with a data part and address part. The elements are linked using pointers and addresses. Each element is known as a node. Due to the dynamicity and ease of insertions and deletions, they are preferred over the arrays



Java Iterator

- An Iterator is an object that can be used to loop through collections, like [ArrayList](#) and [HashSet](#).
- It is called an "iterator" because "iterating" is the technical term for looping.
- To use an Iterator, you must import it from the java.util package.
- The iterator() method can be used to get an Iterator for any collection



Iterator with arrayList

```
3 public class ExampleMain {
4     public static void main(String[] ar) {
5         List<String> names=new ArrayList<>();
6         names.add("Larisse");
7         names.add("Skylar");
8         names.add("Sam");
9         names.add("Lucas");
10        // adding element between Sam and Lucas
11        names.add(3, "Joanna");
12        System.out.println(names);
13        // updating element in the list
14        names.set(1, "Skylar Irisa");
15        System.out.println(names.get(1));
16        //remove element from the list
17        names.remove(0);
18        System.out.println(names);
19        //to check if the list contains Larisse
20        System.out.println("do we have Larisse?" + names.contains("Larisse"));
21        List<String> newNames=new ArrayList<>();
22        newNames.add("Chesi");
23        newNames.add("Maelisse");
24        newNames.add("Asa");
25        names.addAll(newNames); //adding a list
26        Iterator<String> itr=names.iterator();
27        while(itr.hasNext()) {
28            System.out.println(itr.next());
29        }
30    }
31 }
```



For -each loop

- For-each is another array traversing technique like for loop, while loop, do-while loop introduced in Java5
- It starts with the keyword **for** like a normal for-loop.
- Instead of declaring and initializing a loop counter variable, you declare a variable that is the same type as the base type of the array, followed by a colon, which is then followed by the array name.
- In the loop body, you can use the loop variable you created rather than using an indexed array element.
- It's commonly used to iterate over an array or a Collections class (eg, ArrayList)



For-each example

```
public class ExampleMain {  
    public static void main(String[] ar) {  
        List<String> names=new ArrayList<>();  
        names.add("Larisse");  
        names.add("Skylar");  
        names.add("Sam");  
        names.add("Lucas");  
        // adding element between Sam and Lucas  
        names.add(3,"Joanna");  
        System.out.println(names);  
        // updating element in the list  
        names.set(1, "Skylar Irisa");  
        System.out.println(names.get(1));  
  
        List<String> newNames=new ArrayList<>();  
        newNames.add("Chesi");  
        newNames.add("Maelisse");  
        newNames.add("Asa");  
        names.addAll(newNames);//adding a list  
        System.out.println("all names ");  
        for(String allnames:names)  
        {  
            System.out.print(allnames+" ");  
        }  
    }  
}
```



Map in java

- In Java, Map Interface is present in java.util package represents a mapping between a key and a value. Java Map interface is not a subtype of the Collection interface. Therefore it behaves a bit differently from the rest of the collection types. A map contains unique keys.
- Maps are perfect to use for key-value association mapping such as dictionaries
- The maps are used to perform lookups by keys or when someone wants to retrieve and update elements by keys



Map

Some common scenarios where you can use map, are as follows:

- A map of error codes and their descriptions.
- A map of zip codes and cities.
- A map of managers and employees. Each manager (key) is associated with a list of employees (value) he manages.
- A map of classes and students. Each class (key) is associated with a list of students (value).



Characteristics of Map

1. A Map cannot contain duplicate keys and each key can map to at most one value. Some implementations allow null key and null values like the HashMap and LinkedHashMap, but some do not like the TreeMap.
2. The order of a map depends on the specific implementations. For example, TreeMap and LinkedHashMap have predictable orders, while HashMap does not.
3. There are two interfaces for implementing Map in Java. They are Map and SortedMap, and three classes: HashMap, TreeMap, and LinkedHashMap.



HashMap

- HashMap is a part of Java's collection since Java 1.2. It provides the basic implementation of the Map interface of Java. It stores the data in (Key, Value) pairs. To access a value one must know its key. This class uses a technique called Hashing. Hashing is a technique of converting a large String to a small String that represents the same String. A shorter value helps in indexing and faster searches.



HashMap example

```
public class MapMain {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Map<String,Integer> course=new HashMap<>();  
        course.put("Java", 5);  
        course.put("database", 2);  
  
        course.put("English", 2);  
        course.put("Kinyarwanda", 1);  
  
        // Iterating over Map  
        for (Map.Entry<String, Integer> e : course.entrySet())  
  
            // Printing key-value pairs  
            System.out.println(e.getKey() + " "  
                               + e.getValue());  
        // Using entrySet() to get the set view  
  
        System.out.println("The set is: " + course.entrySet());  
        // Checking for the Value 2  
        System.out.println("Is the value '2' present? " + course.containsValue(2));  
        // Checking for the key_element 'Java'  
        System.out.println("Is the key 'Java' present? " + course.containsKey("Java"));  
        Map<String,Integer> course2=new HashMap<>();  
  
        course2.put("Java", 5);  
        course2.put("database", 3);  
  
        course2.put("English", 2);  
        course2.put("Kinyarwanda", 1);  
        // Checking the equality  
        System.out.println("Equality: " + course.equals(course2));  
    }  
}
```



10/31/2025

Queue Interface

- The Queue interface is present in java.util package and extends the Collection interface is used to hold the elements about to be processed in FIFO(First In First Out) order. It is an ordered list of objects with its use limited to inserting elements at the end of the list and deleting elements from the start of the list, (i.e.), it follows the **FIFO** or the First-In-First-Out principle.
- Being an interface the queue needs a concrete class for the declaration and the most common classes : **_PriorityQueue, LinkedList, PriorityBlockingQueue**



PriorityQueue

- A PriorityQueue is used when the objects are supposed to be processed based on the priority. It is known that a [Queue](#) follows the First-In-First-Out algorithm, but sometimes the elements of the queue are needed to be processed according to the priority, that's when the PriorityQueue comes into play.

The example below explains the following basic operations of the priority queue.

- [boolean add\(E element\)](#): This method inserts the specified element into this priority queue.
- [public peek\(\)](#): This method retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
- [public poll\(\)](#): This method retrieves and removes the head of this queue, or returns null if this queue is empty.



Example of PriorityQueue class

```
3 import java.util.PriorityQueue;
4
5 public class MapMain {
6
7     public static void main(String[] args) {
8         // TODO Auto-generated method stub
9         PriorityQueue<String> pq = new PriorityQueue<>();
10        pq.add("Google");
11        pq.add("For");
12        pq.add("Google");
13        System.out.println("PriorityQueue: " + pq);
14
15        // Using the peek() method
16        String element = pq.peek();
17        System.out.println("Accessed Element: " + element);
18        System.out.println("The removed Element: " + pq.poll());
19        System.out.println("PriorityQueue: " + pq);
20
21    }
22 }
23
24 }
```



Set Interface

- The set interface is present in java.util package and extends the Collection interface. It is an unordered collection of objects in which duplicate values cannot be stored. It is an interface that implements the mathematical set. This interface contains the methods inherited from the Collection interface and adds a feature that restricts the insertion of the duplicate elements. There are two interfaces that extend the set implementation namely SortedSet and NavigableSet



Example of HashSet and TreeSet

```
1 import java.util.*;
2
3
4
5 public class MapMain {
6
7     public static void main(String[] args) {
8
9         Set<Integer> set1=new HashSet<>();
10        set1.add(4);
11        set1.add(59);
12        set1.add(41);
13        set1.add(34);
14        set1.add(4);
15        System.out.println(set1);
16
17    }
18
19 }
```

[34, 4, 41, 59]

No
duplication,
but the list
is not order

```
1 import java.util.*;
2
3
4
5 public class MapMain {
6
7     public static void main(String[] args) {
8
9         Set<Integer> set1=new TreeSet<>();
10        set1.add(4);
11        set1.add(59);
12        set1.add(41);
13        set1.add(34);
14        set1.add(4);
15        System.out.println(set1);
16
17    }
18
19 }
```

[4, 34, 41, 59]

No
duplication,
but the list
is ordered



Different operations on Set

```
public class MapMain {  
  
    public static void main(String[] args) {  
  
        Set<Integer> set1=new TreeSet<>();  
        set1.add(4);  
        set1.add(59);  
        set1.add(41);  
        set1.add(34);  
        set1.add(4);  
        System.out.println(" the cashcode is  "+set1.hashCode());  
  
        Set<Integer> set2=new TreeSet<>();  
        set2.add(4);  
        set2.add(70);  
        set2.add(45);  
        set2.add(68);  
        set2.add(4);  
        set1.retainAll(set2);  
        System.out.println("The intersection is"+set1);  
        set2.removeAll(set1);  
        System.out.println("the difference is"+set2);  
    }  
}
```

```
the cashcode is 138  
The intersection is[4]  
the difference is[45, 68, 70]
```



Natural Sort using Collections class

- **Collections** class in Java is one of the utility classes in Java Collections Framework. The java.util package contains the Collections class in Java.

```
1
2 import java.util.*;
3
4
5 public class MapMain {
6
7     public static void main(String[] args) {
8
9         List<Integer> list=new ArrayList<>();
10        list.add(4);
11        list.add(59);
12        list.add(41);
13        list.add(34);
14        list.add(4);
15        Collections.sort(list);
16        System.out.println("The sorted list is"+list);
17    }
18 }
19
20 }
```

The sorted list is[4, 4, 34, 41, 59]



Comparator vs Comparable in java

- In the above example, we did the natural sort because String class implements the Comparable interface!!
- What is comparable interface?
- A comparable object is capable of comparing itself with another object. The class itself must implements the `java.lang.Comparable` interface to compare its instances. Consider a Movie class that has members like, rating, name, year. Suppose we wish to sort a list of Movies based on year of release. We can implement the Comparable interface with the Movie class, and we override the method `compareTo()` of Comparable interface.



Comparator vs Comparable in java

- What is a Comparator interface?
- Unlike Comparable, Comparator is external to the element type we are comparing. It's a separate class. We create multiple separate classes (that implement Comparator) to compare by different members.
Collections class has a second sort() method and it takes Comparator. The sort() method invokes the compare() to sort objects.



Comparable Interface

```
import java.util.*;

public class MapMain {

    public static void main(String[] args) {

        List<Integer> list=new ArrayList<>();
        list.add(4);
        list.add(59);
        list.add(41);
        list.add(34);
        list.add(4);
        Collections.sort(list);
        System.out.println("The sorted list is"+list);
        List<Student> st=new ArrayList<>();
        st.add(new Student("Clarisse","Inyange",25,98));
        st.add(new Student("Maelisse","Rwema",23,99));
        st.add(new Student("Brice","Rwema",24,98));
        Collections.sort(st);
        for(Student s:st) {
            System.out.println("the sorted student based on grade"+s);
        }
    }
}
```

```
The sorted list is[4, 4, 34, 41, 59]
the sorted student based on gradeStudent [fName=Brice, lName=Rwema, age=24, grade=98]
the sorted student based on gradeStudent [fName=Clarisse, lName=Inyange, age=25, grade=98]
the sorted student based on gradeStudent [fName=Maelisse, lName=Rwema, age=23, grade=99]
```



Comparable Interface

```
public class Student implements Comparable<Student> {
    String fName;
    String lName;
    Integer age;
    Integer grade;
    public Student(String fName, String lName, Integer age, Integer grade) {
        super();
        this.fName = fName;
        this.lName = lName;
        this.age = age;
        this.grade = grade;
    }
    @Override
    public int compareTo(Student other) {
        // TODO Auto-generated method stub
        if(this.grade>other.grade)
            return 1;
        else
            return -1;
    }
    @Override
    public String toString() {
        return "Student [fName=" + fName + ", lName=" + lName + ", age=" + age + ", grade=" + grade + "]";
    }
}
```



Same Example using Comparator

```
public class Student {
    String fName;
    String lName;
    Integer age;
    Integer grade;
    public Student(String fName, String lName, Integer age, Integer grade) {
        super();
        this.fName = fName;
        this.lName = lName;
        this.age = age;
        this.grade = grade;
    }

    @Override
    public String toString() {
        return "Student [fName=" + fName + ", lName=" + lName + ", age=" + age + ", grade=" + grade + "]";
    }
}
```



Same Example using Comparator

```
import java.util.Comparator;

public class CompareGrade implements Comparator<Student>{

    @Override
    public int compare(Student s1, Student s2) {
        // TODO Auto-generated method stub
        if(s1.grade>s2.grade)
            return 1;
        else
            return -1;
    }
}
```



Same Example using Comparator

```
import java.util.*;

public class MapMain {

    public static void main(String[] args) {

        List<Integer> list=new ArrayList<>();
        list.add(4);
        list.add(59);
        list.add(41);
        list.add(34);
        list.add(4);
        Collections.sort(list);
        System.out.println("The sorted list is"+list);
        List<Student> st=new ArrayList<>();
        st.add(new Student("Clarisse","Inyange",25,98));
        st.add(new Student("Maelisse","Rwema",23,99));
        st.add(new Student("Brice","Rwema",24,98));

        Collections.sort(st,new CompareGrade());
        for(Student s:st) {
            System.out.println("the sorted student based on grade"+s);
        }
    }
}
```

Here, we
used the
object the
class
compareGrad
e

